



PROGRAMMING LANGUAGE AND SYSTEM FOR ENHANCING AI-ASSISTED SOFTWARE DEVELOPMENT

Faculty Pure and Applied Science / Cognitive Systems

Giancarlo Frison

Supervisor: Loizos Michael

December 2025

CONTENTS

Summary	2	The engine	20
Introduction	3	Target deployments	20
Related work	5	Capriccio: the web user interface	20
Neuro-Symbolic AI (NeSy)	5	ARVO: the LLM wrapper	21
Type 1: the obvious	5	Interoperability with information sources	21
Type 2: neural as subroutine	5	REST APIs	21
Type 3: collaborative models	5	Model Context Protocol (MCP)	22
Type 4: compilation-based architectures	5	Combinatorics in Kubrick	23
Type 5: symbolic rules as neuron activations	6	Grounding	23
Type 6: evolutive collaborative models	6	Partitions	23
Automatic program synthesis	6	Repetitions	24
Input as formal specification	6	Filters	24
Input as examples	6	Multi-objective optimization	24
Input in plain English	6	Epilogue	26
Low-code programs	7	Limitations	26
Program synthesis approaches	7	Negation	26
Fully symbolic	7	Naive combinatorial optimization	26
Fully neural	7	Data integration	26
Hybrid	7	Relation algebra: division	26
Symbolic enumerating algorithms	7	Comparison with Answer Set Programming	26
Top-down enumeration	7	Evaluating Kubrick usability	26
Bottom-up enumeration	8	Bibliography	27
Probabilistic guided search	8	Appendixes	28
Large language models	9	Appendix B - LLM prompt engineering	28
Transformers	9	Appendix C - Python program for grocery optimizer	29
Answer set programming	10		
Relational.ai (Rel)	10		
Google Mangle	11		
Background and preliminaries	12		
Anatomy of interaction in programming systems	12		
Affordances and mediators in programming systems	12		
Self-sustainability	12		
Unified mode of programming	12		
Universal programming language	12		
Open authorship	13		
Program and knowledge representation	13		
Homoiconicity: when code is data	13		
Unification: the core of logic programming	13		
Functional programming: immutability and transparency	13		
Relational algebra	14		
Kubrick: language overview	15		
Data representation	15		
The language	15		
Methods: functions and implications	15		
Aggregations	16		
Relational algebra, applied	16		
Core modules	16		
Prelude	16		
Reticolo: Inside the Kubrick engine	16		
Kube: embedded database	17		
Geronimo	17		
Inquiring Kubrick	18		
Core workflow	18		
Runtime processing	19		
OR/AND combinators	19		

SUMMARY

AI tools are the horsepower of computer programming. Frontier models are pushing forward the symbiosis between AI and software systems with the agentic paradigm. While their abilities are applied in high-stake tasks, open-source models are fine-tuned for industry domains to reduce operational costs.

However, when confronted with the complexity of software systems these less sophisticated models quickly consume their cognitive capacity on verbose boilerplate, state mutation and fragmented tooling instead of core problem-solving. Seemingly, high substrate complexity creates barriers that prevent humans from properly supervising applications' development.

For mitigating these issues, I designed and implemented a programming system that facilitates the synergy between automatons and humans on workflow synthesis. It is an attempt to lower the complexity barriers introduced by the programming ecosystem by simplifying the human and agentic experience. Kubrick shifts large language models (LLMs) from fragile imperative coding to an high-level declarative data-programming.

The design embraces open authorship. The principle is simple: let end-users to directly evolve the program whenever possible. It allows them to directly read, tweak and co-author the synthesized logic produced with the modularity of a Jupyter notebook and immediacy of a spreadsheet.

Kubrick brings ideas on how to make casual programming easily without losing expressivity for non-trivial problems. This paradigm is particularly relevant on combinatorial problems where automatons struggle without the aid of specialized solvers. This project advances specific features for solving optimizations while keeping the programming experience as simple as possible.

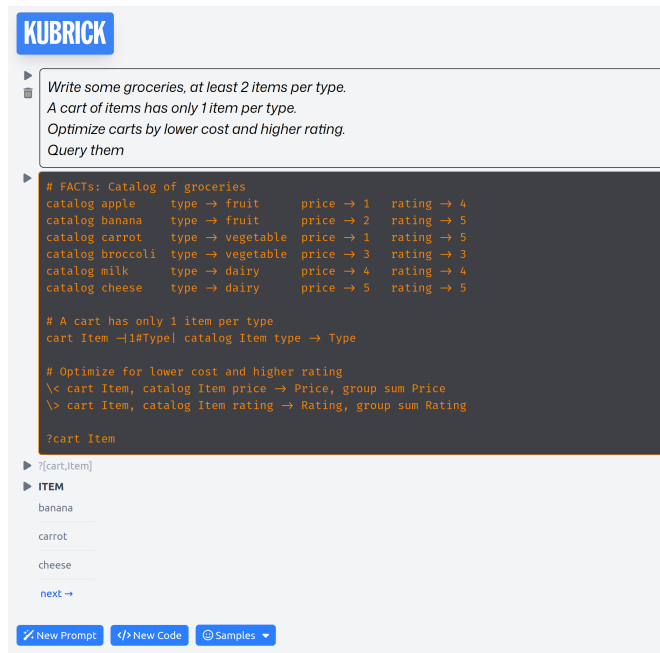


Figure 1: Kubrick prototype is available at <https://gfrison.com/kubrick>

INTRODUCTION

AI tools are everywhere. In every domain it is possible to find processes that can benefit from AI capabilities. If we exclude certain high-risk sectors - nuclear plants, aviation control - AI is already assisting human labour, and the pace of adoption will certainly accelerate. LLMs represent a ground-breaking advancement to generative AI (GenAI) that impact profoundly, and practically all activities of information processing.

Program synthesis is the automatic process of generating programs and thanks to GenAI it has already seen giant steps forward making it easier to build applications. LLMs were initially conceived for language translation tasks, consequently their adoption in

natural language → programming code generation would be unsurprising. After all, the most widespread medium for programming is text, and the programming code, though cryptic for unattended specialists, is fundamentally ideated for being written and read like any other textual form. Is it then reasonable that LLMs exhibits appreciable programming skills as they do on generating natural language texts?

When programmers encounter new code, they actively simulate the program's behaviour in their head and create mental models of its structure and logic. It is less about translating concepts from one representation to another, and more about playing with the code mentally. This is why programming is a logically demanding task, with significant implications on how people interact with programs. This is confirmed by the commonly used debugging tools, syntax highlighting, code formatting, visualization applets; are all there for supporting the brain's reliance on logical capabilities (Ivanova, 2020). Is this one of the reasons why programming generators may disappoint when attempting to replicate human-like understanding and reasoning in code generation?

With AI it is not so bizarre to ship live entire applications with thousands of lines of code automatically just in a single day, but their effectiveness may fall short on accomplishing what prompters want them to do. Complaints regarding GenAI performances are:

- Problems with multi-step reasoning.
- Struggles with mutable states and side effects.
- Shallow code understanding.
- Fail to meet requirements.
- Generate complicated code.
- Despite same prompt, they generate different code.
- Despite different prompt, they generate the same code.

Transformer architectures, the foundations of LLMs, are next-token predictors and they are blamed for those failures because of their architectural design. Before shading light on possible solutions, we need to comprehend what programming essentially means, and I will try by tackling it from two distinct sides: the *problem domain* and the *solution domain*.

Understanding the problem is the most important task for solving it, and if it is more or less difficult to grasp, it is related to its *essential complexity*. On the other hand, the complexity of the solution domain is referred to as *accidental complexity*, and it is introduced by the ecosystem necessary to implement the solution.

The programming system consists of several substrates that include libraries, tools, and external systems. The aggregate complexity increases exponentially the necessary effort to implement applications. While the essential complexity isn't negotiable, it is the duty of software specialists to keep the accidental one at the lower level possible. As often attributed to Albert Einstein:

If I had an hour to solve a problem, I'd spend 55 minutes thinking about the problem and five minutes thinking about the solution

With the increasing complexity of programming system - in terms of tools, libraries and frameworks - creating programs absorbs not only more of the engineer's working memory, but also the resources an LLM needs to solve the same task. This effort is measured by perplexity - how unexpected a token is to the LLM - and it is correlated to the accidental complexity. More accidental complexity, more chances of introducing bugs. Both human and AI show a significant correlation when faced with similar alienating settings (Abdelsalam, 2025).

If we consider AI as an assistant, we need to elevate the role of the programmer to AI supervisor for double-checking if what has been generated satisfy explicit and implicit requirements. Software code is more read than written even without the usage of automatons for coding, and the immense capacity of the generators to flush out large quantity of code can easily saturate human scrutiny, urging the necessary of a clear, concise and easy language for encoding programs.

The attitude on simulating code behaviours confirms that intelligence can't be diverted from the ability to search through a potential infinite combination of concepts and rules. This is in summary the idea of productivity which refers to the compositional generation of optimal propositions from a valid set of grounded statements. How do automatons excel on that? The building blocks of LLMs lack of intrinsic search, though efforts have been applied on forcing recursive reiterations on their conclusions for minimizing hallucinations (Ouellette, 2024). How would be possible to inject intentional search where it is lacking? This is why constraint programming can boost applications' intelligence by mechanizing the generation of

optimized solutions.

The glamoured agentic AI trend put LLMs in the main stage of software workflows exploiting their ability to plan sequence of tasks according to the reasoning and acting (ReAct) paradigm. The limited flexibility for representing dynamic workflows through unchecked text format or free-style Json, has inspired to force agents to produce Python code instead, improving substantially the effectiveness of those agents (Wang, 2024). This achievement reinforces the thesis that a highly expressive and declarative language like the one presented in this project can improve the impact of agentic AI. Many difficulties of automations and developers on generating code might be due to a common root to be found in the accidental complexity carried by the programming system.

RELATED WORK

In this section I will mention current initiatives that share more commonalities with Kubrick. Google Mangle, Relational.ai flagship project ReL, and answer set programming (ASP) paradigm. Though ASP implementations (clingo, for example) may closely align with Kubrick ideas, I will highlight the aspects that remark the unique proposition advanced in this dissertation. For an examination of the GenAI mechanics I will introduce topics like program synthesis and its anchored discipline, the neuro-symbolic (NeSy) AI.

NEURO-SYMBOLIC AI (NESY)

Since the term AI was coined in the '50s, many years of attempts, investments and research have been committed on creating some sort of intelligence that can be as effective as human intelligence. The evolution of AI has seen many changes in paradigms and public attention. The two main schools of thought were the symbolic and the connectionist, and the glamour associated with each of them varied with time.

The pendulum of general excitement swung from the peak reached by symbolic methods in the eighties of last century to the connectionist one which could be easily affirmed as the dominator of the AI scene today. While they have been seen as mutually exclusive, the NeSy movement tells us that combining deep learning with symbolic methods may bring a more effective AI than either paradigm on their own. The principles that govern both approaches seem to be apologetic to each other and very hard to conceal.

The ascent of transformer neural architectures has been accompanied by its integration with symbolic systems even for tackling simple problems. Open protocols (like MCP) are enabling this integration. Frontier models generate programs that can return exact information to them for almost any requested activity. We can safely say that modern AI is the celebration of the NeSy paradigm.

How can neural networks and programming engines be joined together? Henry Kautz classified those methods into 6 categories with the intention of describing their progressive integration. Starting from the more obvious and simplistic configuration to the most sophisticated (Kautz, 2022).

TYPE 1: THE OBVIOUS

The first of them, the basic one, is the
Symbolic $\rightarrow$ Neural $\rightarrow$ Symbolic

where symbols are vectorized for being then processed by the neural network, and then the result converted back to symbols. One might ask where the symbolic processing is in there, since there is no symbolic reasoning at all. As the knowledge representation is symbolic, then it is still NeSy.

TYPE 2: NEURAL AS SUBROUTINE

In the second category - the Symbolic[Neural] type in the Kautz's ladder - the symbolic part is using the neural network, usually as a subroutine leveraging its excellent fuzzy approximation, for serving symbolic computing tasks. An example is AlphaGeometry, a system that finds/generates proof for geometric statements (Trinh, 2024). With it, the neural network is exploited to find shortcuts for accelerating the symbolic search.

In another case - a less commonly known example that fits the GenAI surrounding topic of this dissertation - the use of LLMs through prompting - is type 2 as well. In-context learning, such as zero/few-shot and chain of reasoning (CoT), the prompter is assuming the role of symbolic reasoner while the LLM holds the fuzzy connectionist side. This is why prompting matches the type 2 classification (Bougzime, 2025).

TYPE 3: COLLABORATIVE MODELS

As we climb the ladder of integration, things get more interesting. While in the previous category the neural network is a sub-routine, in type 3 the neural network is a co-routine equally ranking as the symbolic part. The category is depicted as a collaborative model of interaction because the neural and symbolic parts focus on complementary tasks, making the interaction theoretically more performant.

Examples of this approach will be clarified in more detail in the following sections, but I would mention DeepCoder project (Balog, 2016). It generates code by training a neural network to provide a vectorized heuristic of the target program that a symbolic solver can exploit to speed up the search. Other interesting cases include iLLM-synth, by which a probabilistic grammar (pCFG) is generated within the training set of textual description and related code snippets (Li, 2024). The pCFG guides the A* search, pruning out less likely candidates. Another example is a question answering system based on images, where the neural network learns visual concepts; a solver can handle answering questions about the image.

The interactions between modules are not intended to be evolutive since they are not trained end-to-end (Wang, 2022). The neural network is not going to improve the symbolic reasoner (or vice versa). They are complementary but they do not help each other to get better.

TYPE 4: COMPILATION-BASED ARCHITECTURES

The following Neuro:Symbolic $\rightarrow$ Neuro represents a further step towards a blurring edge between the modules. In this category, knowledge is inoculated into the neural network. From this NeSy category on, the symbolic knowledge is compiled into the neural network itself as

embeddings. The domain knowledge is incorporated in the weight parameters of the network during training.

The case of graph neural networks (GNN) is a good example of this approach. The network is trained to predict the missing slot in an RDF triple - consider the DeepWalk algorithm, for example (Perozzi, 2014). More sophisticated ways are those in the messaging-passing algorithms such as the spectral method by which each node encoding is derived from the neighbors recursively (Kipf, 2017). The symbolic structures are transformed for being operated by the neural network into vectorial representations. Similarly to the previous type, the knowledge encoding is just a one-way compilation on which the neural embeddings are not refined via feedback with the rest of the system.

TYPE 5: SYMBOLIC RULES AS NEURON ACTIVATIONS

Proceeding with compilation-based architectures, in the type 5 the symbolic rule/knowledge is encoded into the neural network's architecture but in a different way. Instead of relying on standard activations like ReLU or sigmoid, the neuron is governed by a rule, signaling the validity of its input on its error feedback. This is also referred to as "symbolic integration in loss function".

Variants of this type are the logic tensor networks (LTN) that approximate non-differentiable connectives (and, or, implication and negation) and quantifiers (forall, exists) into fuzzy operators (Badreddine, 2022). With this, rules can be embedded into neural weights.

Considering LLMs, this type can fall into the case where the model is fine-tuned against a specific task that involves program synthesis. The model is not just prompted, but it is trained to match programming patterns (Wang, 2022).

TYPE 6: EVOLUTIVE COLLABORATIVE MODELS

Finally, the last one is described by Kautz as "*the greatest potential to combine the strengths to combine logic and neural based AI*".

It is type 6 referred to as "Full hybridization of neural and symbolic components", the *Neuro[Symbolic]*. With a more holistic definition of this category, we can say it is the closest form of NeSy to the famous Kahneman's System 1 and System 2, where the system 2 coincides with the deductive and compositionally driven refinement while system 1 is the intuitive and fast module linked to pattern recognition (Kahneman, 2011).

Unlike the others, type 6 implies a recursive loop of mutual improvement between the heterogenous modules, where both components evolve. The neural part is trained with the guidance of the symbolic engine, which is then refined with the updated knowledge according to the neural feedback. The feedback loop then continues until it converges into acceptable results.

Think of an autonomous driving system where the neural network processes traffic signs and the symbolic engine evaluates based on contextual rules (like verifying a stop

sign coincide with an intersection). If inconsistencies are found - such as the stop sign in the highway - the symbolic part flags the recognition and the neural network re-evaluates the scene (Bougzime, 2025).

AUTOMATIC PROGRAM SYNTHESIS

A computer program is a symbolic representation that can be transformed into wishful outcomes by a processor. The encoding begins with some form of intentionality that can be non-symbolic. The program synthesis must therefore deal with fuzzy expressions on one side and formal logic and syntax on the other. This is the why NeSy approach best suits the generation of programs. NeSy embraces symbolic and connectionists intuitions and symbolic logic for program generation.

Programs could be generated from some form of specification or by imitating some examples, and the way to generate output varies enormously. On the other hand, symbolic engines perform better on enumerating solutions if they are somehow guided by heuristics provided by neural approximators. Below, I will introduce the different settings of PS problems, the symbolic enumeration search, and probabilistic grammar trees.

INPUT AS FORMAL SPECIFICATION

The requirements might be written in formal language, like test cases in the same language of the generated code or an even more abstract logical form. This is typical of the SyGus challenge, where specifications are runnable code snippets.

INPUT AS EXAMPLES

Rather than providing a strict set of requirements the program should adhere to, the generator is tuned on a few pairs of *input* → *output* as training samples, and then the learned pattern is applied to an unpaired input. This is called programming by examples. One key detail: the output in these examples isn't the actual code itself; it's rather the result of running the target program, and PS succeeds if the generated program produces the same output as the expected one. This type is named programming by example (PBE) and it is the approach adopted, for example by ARC-AGI, an initiative for testing human-like intelligence in software agents (Chollet, 2024).

INPUT IN PLAIN ENGLISH

These inputs may take the form of high-level specifications written in natural language, such a brief description of what the program should do. An example is the Con-Code dataset, where pairs of description and code allow the training of PS's generators from English statements. Using natural language for defining programs has been indeed one of the hardest types of input to work with. The problem is that human language is ambiguous and fuzzy, while code needs to be exact, almost mathematical in its structure.

using natural language to specify a program is too imprecise for programs of any complexity (Dijkstra, 1972)

The lack of rich and complex language models has been a barrier for this type of synthesis, but with LLMs it is possible to capture nuances that were previously out of reach.

LOW-CODE PROGRAMS

The generated program might be encoded in some general-purpose language that is designed - as the name suggests - to cover a very wide range of tasks. However, PS is usually tailored for narrowed domains where language expressiveness is not the biggest advantage. This is where a domain-specific language (DSL) is usually more appropriate (Ellis, 2021).

DSL brings some other advantages to the process. First, a DSL reduces the range of possible solutions a program generator must search through. DSLs are restricted in scope and purposes, and consequently the search space of possible programs is smaller. In favor of DSL, there is also another point: DSLs tend to be more readable for expert humans. DSLs do not prescribe unnecessary boilerplate code. Rather, they keep only the meaningful parts that actually affect what the program will do.

PROGRAM SYNTHESIS APPROACHES

Before going deep down to the methods used for PS, I would distinguish three approaches for PS: fully symbolic, fully neural, and hybrid (symbolic + statistical).

FULLY SYMBOLIC

The exclusive-symbolic systems rely only on algorithms for searching solutions without the support of statistical methods. Basically, they do not take into account any learned pattern for speeding up the synthesis. Those methods usually fetch the entire search space, and when the problem can't be narrowed in a small number of possibilities, the task might turn out to be very slow or even unfeasible.

FULLY NEURAL

On the other end, statistical methods usually count on generalizations offered by neural networks, and thanks to their universal function approximation they tentatively map the input to the final program. Those methods have some drawbacks related to the massive amount of data they require for obtaining convincing results (moreover, in many cases those datasets are synthetically generated by the same models adopted for reversing them into programs. Unfortunately, it introduces the self-selection bias, the kind of problem where the training data is not actually a reliable sample of real-case scenarios).

By their intrinsic nature, purely statistical methods lack the precision that usually is demanded for PS. This is a major issue since the output must run on a formal interpreter. To compensate, post-hoc filtering on multiple program generations is necessary. It consists of generating multiple

candidate programs and filtering them afterward. Usually, it degrades performance with consequent defeat of real-time systems.

HYBRID

An attentive observer may notice that the two paradigms above actually complement each other: while neural networks excel at approximations and fuzzy selections, symbolic methods perform best when nesting and composing primitive operations. So why not combine them by exploiting their strengths and mitigating their weaknesses? Hybrid methods attempt to do just that, but since they are fundamentally different, making them work together is not an easy task.

The boundary where one system ends, and the other begins varies on the method adopted by the authors. Generally, an appropriate metaphor for NeSy could be the one that emphasizes the duality of reason/intuition of Thinking Fast and Slow. It basically describes how well human (slow) reasoning integrates with the (fast) intuitive mental module. Similarly, in PS symbiosis occurs for facilitating the search of the target program: the symbolic clockwork scans the possibilities while the neural module suggests the intuitions to search more efficiently.

SYMBOLIC ENUMERATING ALGORITHMS

What mainly distinguishes symbolic from statistical methods is the presence of enumerative algorithms that represent one of the workhorses of the entire program synthesis. Each time the PS generates a candidate program, it is going to be validated to ensure correctness with the DSL syntax and the specifications[verification].

If the verification fails and the candidate does not satisfy the specifications, the system has just found a counterexample – an input on which the program produces an incorrect output that could be added in the history of failed trials for improving search on the next iterations. This is basically the process named counterexample guided inductive synthesis (CEGIS) and it is applied not only on PBE but also on specifications settings.

As previously mentioned, the fast and intuitive module is deployed to guide the search of candidates, but learning to search works best when it exploits existing search algorithms already proven useful for the problem domain of interest - for example, AlphaGo exploits Monte Carlo Tree Search, DeepCoder uses also Satisfiability Modulo Theories (SMT) solvers. Which kind of searches are usually employed? A distinction should be made among the different approaches I've seen in the surveyed methods.

TOP-DOWN ENUMERATION

The top-down paradigm shows how a high-ranking problem could be decomposed in smaller parts that are nested or connected. In programming, top-down algorithms are the deep first search, its companion breadth first search or the more generic dynamic programming. Those algorithms are recursive since they unfold into branches that

have a similar structure, but they suffer from a big problem.

Many derived branches ultimately prove irrelevant to the optimal solution. If we can infer which subproblems are worthy of scrutiny, we can significantly reduce computation time. This is why we need a more sophisticated way than the ones mentioned above. The A* search combines backtracking search with heuristics that helps to find the right path (Li, 2024). Where do heuristics come from? Of course, from a probabilistic learned model. This is the intuition at work! With top-down enumeration, the model could be called just before the search and just once, since its output includes the elements to drive the search without further inquiries.

Because search branches operate independently - a property called Markovian - top-down methods work well for supervised learning. Consider a target output: a program represented as an abstract syntax tree (AST) is essentially a hierarchy of operations that unfold from top to bottom. If we can encode the AST in a neural network, it should somehow reflect inevitably a tree structure we can use to discriminate against relevant branches to examine.

BOTTOM-UP ENUMERATION

In contrast, bottom-up searches do not offer the same Markovian characteristics. Bottom-up search builds programs from smaller components, and the correctness or usefulness of a program's chunk might only become apparent when it's combined with other segments later in the process.

What might be more appealing in bottom-up settings is that the process of building programs is closely related to the intuition that a human programmer has when he writes small functions first and then combines them to get the desired solution. This differs from the top-down approach, where you start with the big picture and break it down. Instead, bottom-up focuses on solving smaller, manageable problems first and then assembling them into more complex solutions.

Because the search starts at the bottom of the AST tree and moves its way up, every sub-program it generates is already executable. At any stage of the search, any sub-problem has always a concrete value attached to it, which helps to assess how well it combines with other code segments (Shi, 2022). So, how does the model contribute to the search? Let's talk about some probability-based methods that help those enumerations.

PROBABILISTIC GUIDED SEARCH

Most practitioners agree that heuristics are useful for finding the right solution, and these shortcuts are based also on observed patterns. It is evident that not all programs are equally likely - some patterns appear more often than others. There definitely is a bias in the way programs are written. For instance, it is self-evident not a single program uses both $(> \times 0)$ and $(\leq \times 0)$ - that would be simply absurd.

Another suggestion for inferring which operations may be

involved comes from an analysis of the training data labels (on PBE settings). Is there some kind of alignment on the output elements, for example sorting? If so, then most likely there is a sort of command in the target program (Balog, 2016).

■ Probabilistic grammar tree

Earlier, I exposed some reasons for using DSLs as target programs. One advantage is that DSLs come up with a well-defined context-free grammar (CFG) which is a formal way to define how languages are structured. A CFG is defined by a set of:

- terminal symbols - those atomic elements that can't be derived from other elements.
- variables - to be computed by the symbolic engine and rule of production (non-terminal elements).
- rules of production - the core of the CFG because it is where the generation can unfold the expressiveness of the language and cover myriads of use cases.

The rules determine the language's expressiveness, allowing it to generate countless use cases. Think of it when forming sentences in English: not every word sequence is valid, and a CFG strictly defines which combinations are allowed. This rigidity contrasts with neural models, which learn from examples and work non-deterministically rather than following hard rules.

Probabilistic context-free grammar (pCFG) is an extension of CFG where each rule is associated with a probability that reflects the chance of choosing that rule when expanding the program in a sequence of terminals. Those probabilities (or weights) are statistically derived from a series of programs linked to a specific task.

■ Attribute vector

Another way to learn heuristics from training data is the approach used in DeepCoder (Balog, 2016) where the system uses a vector to represent the probability of programming features present in the target program. The encoded features depend of course on the DSL adopted for the task. For giving some examples, it might be present features such as:

- Unary math operations (like: `-`, `abs`, `sqrt`).
- Binary arithmetic (`+`, `-`).
- Sequence helper methods (`head`, `tail`, `take`, `drop`).
- Set functionalities (`intersection`, `union`, `diff`).
- Filters (`>`, `<=`, `=`).
- Tailored compositions of functions (`mergesort`).

The (probabilistic) attribute vector is then used to elicit discriminatory decisions when the enumerator searches for viable solutions.

■ Argument selector

The two types of statistical models above fit greatly with top-down search since they capture the overall structure of the target program. In contrast, systems such as Cross-Beam (Shi, 2022) take a bottom-up approach. Instead of reasoning from the top level, the model decides how to combine previously explored operations to build new program components.

As we have seen, not only are the programs kept in history but also their execution results. In this way, in the following iterations those program/value pairs are then used to generate more complex candidates, until the desired solution is found. In this way, promising combinations are prioritized rather than exhaustively enumerate all possibilities.

■ Branch selector

In the neural guided deductive search (NGDS), the model is intended to rank potential extensions of a partial program. It takes the input-output examples and the current candidate program, then assigns weights to different branches the search algorithm could explore (Vijayakumar, 2018).

At the beginning, the candidate program is of course empty, and there starts the divide and conquer topdown approach by building the solution step-by-step from the root of the AST. When the enumerator reaches a branch, the model suggests the most promising path to follow - guiding the search toward a valid solution.

■ Auxiliary construction

What if the neural model doesn't just help to speed up the search by giving proper heuristics, but rather it generates new supporting information, helping the symbolic engine to discover a better path to the correct program? This is what has been proposed in AlphaGeometry with the purpose of creating proofs to verify geometrical theories (the reader might find mathematical proofs a bit out of scope with the topic of this dissertation, but the Curry-Howard correspondence equates propositions with types and proofs with programs. That means when we are talking of geometrical validity proofs, we are describing programs that derive consequences from axioms to the target state).

LARGE LANGUAGE MODELS

LLMs are born for natural language processing (NLP) tasks, particularly for generating text. They are autoregressive models, meaning they predict the next word in a sequence of words provided. Neural network techniques for natural language were discovered within the advent of deep learning. The kind of problems interesting to solve with neural NLP include translation, summarization, and question answering.

The early successes of NLP could be found in the statistical NLP wave, where NLP was applied with statistical tool for improving its efficacy. Then it was time for neural networks and their capability of universal function approximators to

give new energy to NLP development. The marriage between neural networks and NLP was celebrated by the ubiquitous Word2Vec and its way to transform words into embeddings that paved the way to numerous achievements. Vector embeddings are a connectionist representation of words with some peculiarities that make them so useful on differentiable processes, the ones at foundation of neural network success. They do not merely give a grade of similarity (coffee is closer to tea than to Airbus, for example), but they offer also algebraic capabilities such as the equation:

kind $\Rightarrow$ queen / man $\Rightarrow$?

where the relation between king and queen is used to determine the correspondent slot associated with man.

In text generation tasks many ideas has been proposed (e.g.: Seq2Seq) but the neural network tool used for that (LSTM) suffers to some degree of vanishing gradient problem which stands for limited memory capacity that prevents the assimilation of complex patterns. That issue was tackled by an advancement that gave birth to the third wave of NLP with the adoption of transformers. Transformers represent a unique class of neural architectures that effectively address the vanishing gradient problem.

TRANSFORMERS

The transformer architecture has been presented at NeurIPS (Vaswani, 2017) and it lays at foundations of LLMs that represent the current state-of-the-art (SOTA) in many NLP tasks. How does the transformer work?

As mentioned in the previous paragraph, LSTM suffers from vanishing gradients over long distances, limiting the capabilities of neural architectures based on LSTM (and its related ones). Transformers have solved that problem with the positioning and the self-attention mechanisms, allowing the scalability of networks into hundreds of billions of parameters. The unprecedented scale of those models tackles not just one learning problem (the so-called narrowed AI) but rather they acquire several skills, leading some specialists to claim that LLMs may be candidates for general problem solvers (Ge, 2024).

Transformers are composed of two main parts: the encoder and the decoder. The former is intended to capture relations among input tokens, while the latter will use the encoder mapping for capturing the relations within input and output samples and driving the generation of the output tokens. For example, in machine translation, the input will be an Italian sentence "buongiorno, come va?" and the role of the encoder is to create a representation of the input text. The decoder has the role to generate a stream of tokens that for the assigned task correspond to the English translation. That's means the output would be "good morning, how are you?".

Though the role of encoder and decoder modules is clearly distinct as the translation task may suggest, the processes in both are quite similar and differ only on how self-attention (SA) is calculated.

Encoder

The input text is tokenized and replaced by respective embedding (as any deep NLP task) but then the innovation starts with the positioning.

Positioning Positioning, as the name suggests, encodes the token embedding with its relative position in the sentence. As said, LLM are training against an enormous corpus, and as we know, neural networks deal with differentiable numerical vectors of fixed size. How can an unbounded integer (the word position in such long texts) being transformed into an appetible format for gradient descent algorithms?

The position is converted into any arbitrary vector alternating the $\sin(f(\text{token}, \text{positioning}))$ with the cosine of the same function f . The resulting vector could be seen as a wave that preserves a precise and differentiable relative distance information with the other peers, the other tokens present in the input.

Self-Attention The self-attention (SA) mechanism allows us to establish relevancy relations among words in a text. It is not just a coreference resolution aiming to group references to the same entities, rather it detects attendance patterns between all words. In this way, SA gives context for each word in the sentence, paragraph, document where specifically is referred.

SA is implemented by integrating three different elements that are all differentiable and backpropagated: queries, keys, and values.

- The query is processed for each token and takes as input the embedding and its positioning. Since it is referring to a single token, it could be performed in parallel. Queries are processed with their own set of parameters, the same for all tokens.
- Keys are calculated for each pair of tokens; therefore, they represent the bulk of computation in SA. The result is a soft-max vector between the token query and the other tokens.
- Values are then calculated between embeddings + positionings and their respective keys defined above. Values represent the output of SA.

SA processes might scale by chaining multiple SA (multi-head attention) to capture farther relationships among tokens.

The positioning layer is added to the SA layer for the residual connections layer. In this way, the position, the word embeddings, and the SA are included in the encoder's output.

Decoder

The decoder is intended to take part in the generation of the output text. In the sample case of machine translation, the decoder will consider the encoder output but with an eye on the output domain (different language). The decoder includes the SA as well but with the peculiarity that for each token generated (the Start of String is given as input seed token by default) and the respective query, the

keys for generating the token's values are retrieved from the encoder's ones. The parameters used for processing the decode SA are different from those present in the encoder, giving the necessary decoupling from the input and the expected output. This is the first difference between the encoders.

The second difference consists of the way the output token just generated is used to feed the SA in a recursive fashion. The decoder sustains the generation of new tokens until it reaches the generation of end of stream (EOS) that terminates the cycle, and the generated stream (text) may be returned as output of the entire transformer. The decoder's SA could be chained just as in the encoder, for improving the sophistication of generated stream of text.

ANSWER SET PROGRAMMING

The purpose of ASP is to generate combinatorial solutions over an initial base of facts and rules. What differs ASP from other logic programming languages like Prolog is that ASP aims to generate stable models that satisfy all generation principles and constraints. An ASP runtime will terminate only when no new solutions will be possible to generate from those already found, or when no solution exists. This is the crucial difference from Prolog for example, which computation is initiated by a query that can be validated/invalidated or can bind proper variables to atoms in the database, rather than generating multiple instances of facts. ASP is then suited for combinatorial problems, and its language reflects this purpose.

The specific affordance is given by the choices, a syntactic feature that describes how single atoms may spark the ramification of the solutions. Consider the following code:

```
color(red; green; blue).
node(X) :- edge(X, _); edge(_, X).
{ hasColor(N, C) : color(C) } = 1 :- node(N).
:- edge(A,B), hasColor(A,C), hasColor(B,C).
```

The third row establishes the generation of multiple solutions by indicating that each node will ramify into as many solutions as the color cardinality, and in each of those solutions, the node will have different colors. This is the generational part of the program. The last row instead will prune out all solutions on which adjacent nodes have the same color. This is defined by using the same variable C for both nodes, here the engine will attempt to unify the nodes with the same color, and if it occurs, the solution is discarded.

RELATIONAL.AI (REL)

Rel is the language of the Relational AI platform, a knowledge graph management system for building adhoc services in the Snowflake ecosystem. As the name may suggest, Rel is a relational logic programming language, and it incorporates concepts like relational algebra as previously discussed. There are several overlapping concepts with Kubrick, both attempts to minimize the impedance mismatch between host and query languages, and they were influenced by Datalog for representing data in its more basic form (Aref, 2025). An example of Rel code is the pairs of shortest path example:

```

def APSP({V},{E},x,y,0)
  : V(x) and V(y) and x = y
def APSP({V},{E},x,y,i)
  : i = min[{(j): exists((z) | E(x,z) and
APSP(V,E,z,y,j-1))}]

```

Rel shows a similar pattern matching Kubrick, and the lines above could be interpreted in two distinct phases. In the recursive fixed-point definition, the path between x and y is zero if they are the same. Otherwise, it is the minimum of those between x and the distance with the paired node.

Rel adopt the Graph Normal Form for dealing with record properties, which is the most disentangled form among the normal forms - for storing document-based records. In this way, the underlying format is strictly adherent to the RDF principles. Moreover, The GNF adds the notion of temporality to the CRUD (create, read, update, delete) operations to guarantee traceability of database's changes (Frisendal, 2022).

The kind of data intended to be managed in Rel is the one defined by RDF standard. Data consists not only of atomic types (numbers, Booleans and strings) but also URIs, which are used to uniquely identify entities. By that, they say things, not strings motto claimed by Relational.ai to indicate that contextindependent concepts are preferably managed in Rel. This is another similarity with Kubrick; it is then important to notice that the publication of Rel is posterior to advanced stages of Kubrick development.

GOOGLE MANGLE

The Mangle language is a recent proposal by Google Research for a logic programming language for relational algebra. As seen in the previous projects, the Mangle authors acknowledged the sole query purposes of languages interfaces exhibited by databases, and they proposed merging programming with querying paradigms under the same umbrella (Saxena, 2025). Initially committed on GitHub in 2023, it got traction from public announcements while this thesis was progressing into its final stages in August 2025.

Basically, Mangle carry on all selling points of Kubrick:

- Recursive rules. Ideal for tracing dependency trees or access hierarchies
- Uniform data access. Pulls facts from files, APIs, or DBs, treating them as one dataset
- Aggregation & function calls. Supports operations like count, sum, and even external function calls, so you can blend logic with custom business rules
- Practical syntax. Built to be developer-friendly for those used to Go or imperative programming

BACKGROUND AND PRELIMINARIES

ANATOMY OF INTERACTION IN PROGRAMMING SYSTEMS

If designing a system to solve problems is hard also due to the complex intricacies of the programming ecosystem, the low-code wave that it is persisting since the advent of computing *should* be the recipe for creating software solutions. The low-code (and its extreme sibling “no-code”) has begun with the promise to lower the barrier for programming, enabling a wider range of people to participate in the software development process without needing extensive knowledge of the programming system. The slogan “*No Programming is Required*” was coined by advertisers since the '70s for emphasizing the ease of use of software system on sale.

AFFORDANCES AND MEDIATORS IN PROGRAMMING SYSTEMS

When an interaction occurs between agents (users, developers, automations) and the system, it is mediated by the affordances the system exposes. Mediation occurs at different levels. It could be described as typing on the keyboard for writing, as well as navigating graphical user interfaces using the mouse. For the human user's perspective, software contains many *mediators*, variously suited to the application final goal (Basman, 2018).

The problem with mediators is that they usually prevent customizations. Often, mediators are close to authorship, which means that any choice made at development can't be overridden by the users. Decisions made by programmers are usually immutable and denied of any alteration.

Take for example GUI interfaces, they are usually fixed, and it's impossible for non-designers to change them. Those mediators perpetuate a hard separation between who creates the program and who uses it. Users have no expectations of controlling changes in the application. One of the programming system goals is to give a feeling of ownership. Having users empowered to change how the software behaves is important, and any product manager is aware of that: for meeting user demands, what is more flexible than allowing users to modify some aspects of the software themselves?

The open authorship principle should allow the adoption of “*open ecologies of function and fabrication*” which basically means the programming system used by the initial designers should be modifiable within itself. Users should be empowered to handle the exception to their workflow, and they should gain the power to experiment - and in principle - replace parts of the system with better versions (Basman, 2016).

For allowing this, I will introduce the concept of self-sustainability and unified mode of programming.

SELF-SUSTAINABILITY

How could it be useful in embracing a low-code approach and lowering the cognitive barriers on programming when agents are unable to modify the application when exceptions arise? Self-sustainability refers to the extent to which a system's behavior can be changed without having to step outside to a lower implementation level. A programming system that embraces self-sustainability allows its inner workings to be accessible from the user level (Jakubovic, 2023). This may not matter so much when users need to develop an almost prototypical case; self-sustainability shows its benefit when there are exceptions to the usual workflow the platform developers could not have thought of.

This feature can open to new interesting possibilities, for example the user to programmer progression. If the system can be modified, it is then possible for a user to become a programmer or at least acquire awareness of how the system works and how it could go wrong. The functional operator does not necessarily rely on developers' consultation, rather he can access it by himself.

If experimenting on live production sounds too risky - and indeed it is - a paired test environment should be provided to allow users to explore changes. What can enable a transportable environment with minimum friction from operations is a unified mode of programming.

UNIFIED MODE OF PROGRAMMING

I think everybody has at least once used a spreadsheet application. One thing that's very clear to users is that the spreadsheet does not have really a separate environment for programming and for use. A spreadsheet can be modified at any time by modifying the data or the formulas it contains. The user sees the results in the cell immediately once the data or the program changes. There is no other way of modifying the behavior of the spreadsheet in a way different from just updating the visual cells.

Software development often involves a hard separation between programmers and users. The application is typically layered with several substrates that encapsulate different implementations and environments. A substrate refers to an underlying layer upon which a feature rests or operates. In programming systems, different substrates represent various underlying mechanisms or interfaces that allow users to interact with and modify the software.

UNIVERSAL PROGRAMMING LANGUAGE

The dichotomy between query and programming languages has been tolerated for the apparent divergency the two paradigm exhibits. Programming involves applying control structures that manipulate data by the host programming language - Java, Python, for example. On the other hand, data is stored in dedicated information sources, and they are retrieved using their query interfaces

- like SQL for example and then passed to the host environment for further processing. Should query and programming languages be so different like they are?

OPEN AUTHORSHIP

To lower the accidental complexity, we need a broader perspective not only on how developers but also how final users might be involved in the development process. With it, I won't discuss whether program consumers will find the application usable - according to standard human-computer principles - but rather how they may be enabled to adjust the application by themselves and to take the role of programmers themselves.

PROGRAM AND KNOWLEDGE REPRESENTATION

Traditional languages like Lisp and Prolog have stood the test of time with great honor. Lisp has inspired generations of new languages, and advancements are still being made in its style. As someone influenced by this legacy, I want to highlight some aspects that I believe are still relevant for modern programming.

HOMOICONICITY: WHEN CODE IS DATA

This strange term identifies the property by which a language representation of programs is also a data structure in that language itself. In a homoiconic language, the code is structured using its own data types. This allows the program to manipulate code as it is just another chunk of data, enabling powerful metaprogramming capabilities: macros. Another aspect is that treating data as code - and conversely, code as data - lowers the barrier for modelling logic. With that, a function call is not represented differently from querying the database. Simple data retrievals are interchangeable with sub-routines invocations. The difference between programs and data is that the former output is computationally derived while data is retrieved from a database, but from the point of view of the programming agent they are semantically equivalent.

Homoiconic languages have the advantage of providing a uniform interface for data and computation. One premise for successful programming systems is the ability to change how the language behaves within itself. This would be achieved by introducing the capability to write macros; they are programming fragments intended to build new programs inside the programming system itself. Though macros are not implemented in Kubrick yet, it won't break any already defined concept in the language and in the runtime once the feature is activated.

UNIFICATION: THE CORE OF LOGIC PROGRAMMING

The founding principle of logic programming is caring for the logic of complex systems, a trait that has never been of secondary importance in software systems. If I've been

asked what is the most important concept in logic programming, I won't hesitate on unification. It is the process to make two or more variables identical by finding a substitution that satisfies the matching pattern (Dincbas, 1987). This is how LP-based systems evaluate queries, bind variables, and perform logical inference. It is a powerful multi-way pattern matching mechanism that facilitates the usage of multi-faced functions from different perspectives.

The pillars of LP are unification as above mentioned but also backtracking, the innate ability to recursively search through the space of possible solutions. Those concepts are what make LP a higher-level programming paradigm: instead of focusing on how to elaborate the expected solutions - like imperative programming does - it is enough to declare what the solution is and let the engine figure out how to find it. LP systems simplify the programming work because it is intended to automate the steps that other systems would require additional work.

As an example, two points trace a vertical line when they lay in the horizontal coordinate are the same as in the statement:

```
vertical (point X Y1) (point X Y2) -> X = Y1 Y2
```

In this case, X should be identical in the first arguments of both points. Unification guarantees in the example that X values applied to both point predicates are the same. The query process implements unification for this purpose.

FUNCTIONAL PROGRAMMING: IMMUTABILITY AND TRANSPARENCY

Lisp on the other hand, is based on a functional programming (FP) basis with great capabilities to define macros, thanks to its inherent homoiconicity that facilitates the definition of meta programs, snippets of code that change the language itself. This is the property derived from using the same basic data structure (sequences) both for writing programs and for their internal representation. Advanced self-sustainability has been enriched by the modern Racket language that carries on the Lisp tradition till now.

Lisp is an example of functional programming (FP), a paradigm which is the closest form of writing mathematical theories by means of programming. Thanks to sophisticated compilers, it is usually inadmissible to code unrepresentable structures, and this facilitates enormously the development. Being founded on math principles is not just a metaphor. A branch of math called category theory is widely implemented for lowering accidental complexity while ensuring correctness.

FP promotes some principles that help simplicity modularity and maintainability. Usually, mutable states are prohibited unless explicitly declared, and agents are relieved of the burden to follow the changes a structure might be affected along the flow. Composition is fundamental for aggregating simple functions to obtain complex ones and reuse that function for even more variegated applications.

Those legendary languages come from the ancient history of computer science, but they are not currently adopted by the industry. I think the flashing intuitions behind those

languages fit grandiosely with modern needs, and this thesis is intended to demonstrate it, or at least, to attempt to.

RELATIONAL ALGEBRA

Codd's relational algebra defines the operations that tell the database how to derive the expected result from existing data. A relation is an abstract concept. Relations link values like edges do in a graph, or as tables do in a relational DB (RBDMS). It's based on set theory and performing operations on relations for producing new relations in output (Codd, 1970).

The operations expected by Edgar F. Codd are those from set theory like union, intersection difference and cartesian product. Moreover, relational algebra describes the primitive operations necessary for performing all queries (Codd, 1970). They are:

- **Selection.** Filters data based on conditions. E.g. filters the “students with attributes of age greater than 30”
- **Projection.** Selects specific attributes from a relation. For example, retrieve just name and age out of Students.
- **Join.** Matches related tuples from different relations. E.g. to retrieve all combinations of students and courses they are enrolled in.
- **Division.** Filters relations that satisfy all values (forall in Prolog) in another relation. Example: to retrieve students enrolled in all courses.

Aside — Object relation mapping, a panacea for relational/object mapping?

I mentioned in the introduction the discrepancies between the host and the query language in applications that require external data. Those idiosyncratism are mediated in various way and in the object-oriented world, this has been conducted by ORMs (object relation mapping) frameworks that transform the relational nature of data (e.g., tables and rows) into objects and classes. ORMs brings some intuitive advantages in their initial adoption, but soon ORMs reveal itself a slippery slope that can trigger more problems than it solves. ORMs obey to the case of diminishing returns and while ORMs provides initial benefits with low investments, it is easy to recognize that covering all cases of adoption will lead to more investments while benefits are diminishing. Moreover, fetching data by adopting object-oriented programming (OOP) principles may give the illusion of simplicity that soon clashes with the relational nature of the data. Database - and the RBDMS in particular - are based on set theory and tuples; not inheritance, identity, encapsulation nor polymorphism (Neward, 2006).

KUBRICK: LANGUAGE OVERVIEW

Kubrick is a programming language and runtime. The language is the front-end of the application engine. By writing programs in Kubrick, the user can define facts, methods, complex workflows, and queries. The system can target different execution environments such as web browsers and cloud servers. The system is intended also to advance a proper user interface (UI) for programming with the assistance of LLMs and incorporate the insights accumulated in the previous sections in regards of programming experience and ergonomics. This dissertation is not only the result of research and self-education, Kubrick is also the epilogue of more than one year of working and 22 thousand lines of Scala code briefly described in the following sections.

DATA REPRESENTATION

Central to the language is the concept of named tuple which allows you to define document-based data structures. Primitive types are integer and float numbers, strings, Booleans. Not limited to literals, also URIs are supported as prescribed in the RDF (resource description framework). In the record example, a prefix is defined for like in this record:

```
prefix sc: <https://schema.org>
prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
sc:car id-123
  rdf:label → Enzo
  sc:manufacturer → Ferrari
  sc:price → [250000, eur]
```

A prefix is a shorthand notation to define a namespace for URIs. The record's head is a URIs equivalent to `<https://schema.org/car>` and the following items are name/value pairs. The price is a composite value that includes both the amount and the currency. The named tuple is isomorphic with widely known data formats like Json, making it possible to deal with virtually all interchangeable formats with appropriate transformations. In Kubrick, the computing results might be non-unique. This is representable also in the data as choices:

```
house green;red;blue;yellow;ivory
```

With it, there are 5 houses with different colors (Zebra puzzle).

THE LANGUAGE

Kubrick is a declarative programming language based on relational algebra inspired by Lisp and Prolog. From Prolog, it inherits the logic programming paradigm, its multi-way of querying, and the unification principle.

Variables notation is also represented in the same way; all capitalized words are treated as untyped variables. A variable brings none, one or more values. Then could we say that Kubrick is a nondeterministic engine? Yes, because multiple values can be bound to a variable, but at the same time they are not probabilistic. Variable materialization consists of returning a series of choices that satisfy the

query.

Choices might be bound by other variables. When it is so, they can't be discerned from one another. If a choice in a variable fails in a subsequent call, all the entangled variables' values will be discarded as well. This is essentially the core mechanism of unification, and the principle will be unfolded in the following sections.

Similarly to Lisp, predicates may denote also functions that return a single unnamed variable. Consider the following math function:

```
f(x, y) = x * 10 - y
```

If implemented in Prolog, we should write:

```
f(X, Y, Out) :- Out is X * 10 - Y
```

Which is verbose and boring. A more functional style improves readability, and it is achievable with Kubrick:

```
f X Y = - (* X 10) Y
```

Both styles are possible in Kubrick, but of course the more functional one promotes compositionality while the Prolog style is intended to leverage the unification mechanism. The syntax is reduced to the minimum and can be used to express not only facts but also methods.

METHODS: FUNCTIONS AND IMPLICATIONS

Named tuples serve as well as interfaces for methods. They are composed of premises and heads. Those may be functions - as in functional style of programming - or implications, if we treat them as Prolog predicates. The difference is that functions are invoked with materialized arguments and return to a single column, while implications work over the variables passed in the head.

A function is defined if it follows the pattern `HEAD = PREMISE`. An example of function is the SoftMax:

```
softmax X = A → (exp X), / A (group sum A)
```

The column of values named with X will be elevated to exponential and then divided by the aggregated sum of such exponentials to produce the softmax output.

If written as implication in Prolog style (`HEAD → PREMISE`), it will be necessary to define the predicate with three arguments, the third will be materialized as implication's output:

```
softmax X Y -|
  A → (exp X)
  Norm → (group sum A)
  Y → (/ A Norm)
```

Borrowed from Prolog, implications allow multi-directionality in the query mechanism. In this way, the same implication may serve multiple combinations of inputs. For example, you may ask for the softmax of a column or to check whether the provided column Y is a correct softmax output.

AGGREGATIONS

Performing aggregative functions over the entire set of choices of a variable is a basic feature for any useful reporting tool. Unlike Prolog, Kubrick has a more direct and ergonomic approach for attaining aggregations. Let's illustrate it with an example. We have some facts:

```
do roy drink_coffe hours → 3
do roy watch_video hours → 2
do max drink_coffe hours → 3
do max watch_video hours → 2
do eli meeting
hours → 1
```

For obtaining the total hours of activities grouped by user in Prolog (assuming the facts are re-written in Prolog):

```
total_user_hours(User, TotalHours)
:bagof(H, A^do(User, A, H), HoursList),
sum_list(HoursList, TotalHours).
```

The `bagof/3` implements the “group by” logic. It collects all hours (H) for the User. The activity variable (A) is existentially quantified, and the results are stored in `HoursList`, which is then aggregated by `sum_list/2`.

In Kubrick, it is much simpler:

```
total_user_hours User =
do User A hours → Hours
group sum Hours groupBy → User
```

RELATIONAL ALGEBRA, APPLIED

Not all elements prescribed by Codd have yet been implemented in Kubrick. While selection and join are clearly at the foundation of the language, the projection principle is a more subtle feature that may manifest itself in different ways. If a specific projection is requested, it should be unified with a specific variable:

```
user id-123 name → Name
```

In this case, `Name` is a projection of user's name. In SQL, the project is declared in the SELECT statement: `SELECT name FROM user`, or it could be achieved by with OOP patterns like `user.name` where `user` is an object with attributes. While this kind of projection is not implemented in Kubrick, I think it would be interesting to have it to make it easier to deal with attributes.

The division principle instead is not implemented at all in the language. In Prolog it is offered by the `forall` functionality, which succeeds when all choices of a variable succeed. In the default mode instead, an invocation succeeds when at least one choice is validated.

CORE MODULES

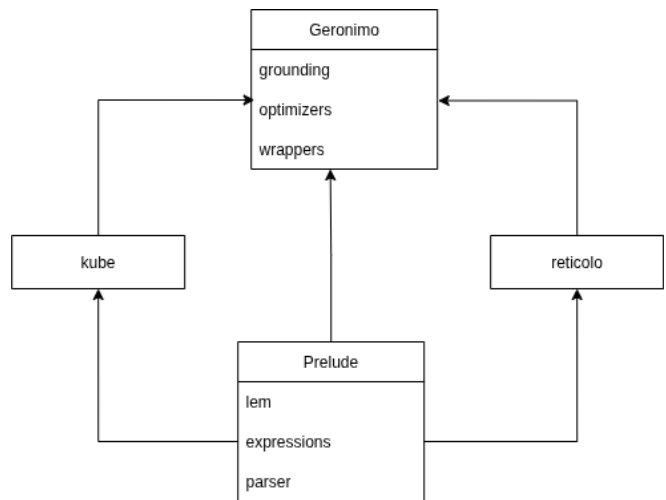


Figure 2: Static dependencies of core modules
Kubrick's core is composed of 4 modules:

PRELUDE

- **LEM.** Fundamental recursive data structure composed of:
 - `Lil` – empty LEM. It is adopted to respond to monoid compositions.
 - `L1` – single value wrapper.
 - `Bin` - it has left and right attributes, both LEM type. The kind attribute is one of the following: `pair`, `or`, and `and`. The `pair` is for indicating paired LEM in named tuples. The others indicate the logical relation between the two sides.
- **Expressions.** They distinguish the language semantics of expressions, such as functions, implications, native operations, aggregations, etc.
- **Parser.** The submodule responsible for parsing program text into a set of expressions and facts.

If in LEM there are only empty, single, and binary representations, how is it possible to encode unlimited sequences, for example? A sequence of 1 to 3 is just:

```
Bin(and, 1, Bin(and, 2, 3))
```

With this basic common structure, it is quite easy to sample all the different kinds of data the system can deal with.

RETICOLO: INSIDE THE KUBRICK ENGINE

As we've seen, a query is the call to action for Kubrick. When the execution begins, an empty context is initialized. A context is a set of disentangled clusters, and each cluster is a table of entangled variables with respective choices. *Reticolo* in Italian means “lattice, grid”, and it refers to the set of grids that are carried out at each single computational step.

A grid is a table where variable identifiers are the table's head, and the values as rows. A grid or cluster is a single row-biased array where a column (variable values) are those cells in the array equally interleaved by the length of a single row, which is constant along the array. The cluster

can perform the following operations:

- **strict join:** two clusters with common variables are merged, the matching rows are kept, the others are discarded.
- **weak join:** two clusters with common variables are merged, the matching rows are kept, the others are Cartesian-ly expanded with other rows.

KUBE: EMBEDDED DATABASE

Unifying data and computation irrevocably involve letting agents specify some facts of reality (data) as code along with methods. That's permitted by an internal database that stores named tuples. It consists of a set of indexes designed for fast retrieval, and they carry chunks of information that form a hierarchical structure. More specifically, the database could be seen as a hypergraph. Graphs are a particular type of data representation on which single nodes - the minimal unit of manageable information - relate to others through relations. Relations - or edges - in graphs may connect two nodes or may connect a multitude of nodes. In the latter case, they are called hyper-edges.

Hyper-graphs (set of hyper-edges) are not a naive data representation. They are more common than the name suggests. Relational databases are a form of hypergraphs, for example. If you consider the table name as the label of the edge type, and each row as an edge of the specified edge type, the cell would be the single node. The main difference one might notice is that in a graph, nodes are uniquely identified via an URI, while in relational databases the uniqueness is defined via normalization.

Not just facts, but also programs are stored as hyper-edges. For example, a function has a head - that acts as interface/definition, and the premise - that is the actual body of computation in a function. Both those parts are stored in a separate database, by leveraging the same storing mechanism adopted for facts. The reason for a custom and specific database rather than using other forms of embedded databases is related to the need of streaming potentially huge number of data chunks for *combinatorial problems*. The database storage as represented here is a very light-weight structure that minimizes the memory footprint of each database-version/solution¹.

GERONIMO

This is the module that exposes the high-level API the clients may use, and the higher-level logic on top of reticolo and kube modules. The entire framework lays its foundations on functional, asynchronous streaming libraries like cats-effects. With it, the system is open to invoking concurrently internal and external services while keeping control over computation and errors. The module includes the following sub-packages:

- **Grounding.** The policies for repetition, partitioning, and combinations of them.
- **Optimizers.** Strict and pareto optimizations.
- **Business wrappers.** Decorators (design pattern) for language expressions.

¹More information can be found in the 2024 patent <https://gfrison.com/patents/hypergraph-store-query>

INQUIRING KUBRICK

The execution of any data processing task in Kubrick starts with a query. Syntactically, the definition of a query is the same as in Prolog. A query is one or more data/method invocations prefixed by a question mark:

```
?softmax 4;3;6;1;2;6;4
→
0.0577
0.0212
0.4263
0.0029
0.0078
0.4263
0.0577
```

The engine inspects the tuple (or the set of tuples), searches for methods or facts that correspond to the query. If there are variables in the query, the system will attempt to unify them. The possible ways to inquiry Kubrick are:

- **Factoid query.** Factoid queries are intended to retrieve a variable that may fill its slot from facts or methods. In the query `?(hasProperty sky X)`, `X` is a variable, and it will be materialized in a way that the resulting edges are justified. A variable is replaced to make the query statements true according to the data and the methods defined. Since a variable is a collection of choices, the replacements are applied according to the variable's entanglements. If it is not possible to find any valid value for the replacements, the query fails.
- **Confirmation query.** The answer succeeds or it fails to a precise query that does not hold any variables. In the query `?(hasProperty sky blue)`, there are no variables, and the system will check if the statement is true based on the available facts and methods. The query may succeed if the statement is validated by direct existence of such fact or by derivation from methods.

CORE WORKFLOW

The core offers the API for compiling text into a union of facts, methods, optimizers, filters and queries, which represent the compiled record of a Kubrick program. With it, it is possible to run an already present query or merge it with another compiled program that could be just a separate query over the previously compiled one. In this way, there is a separation between compilation and execution, described in the following diagram:

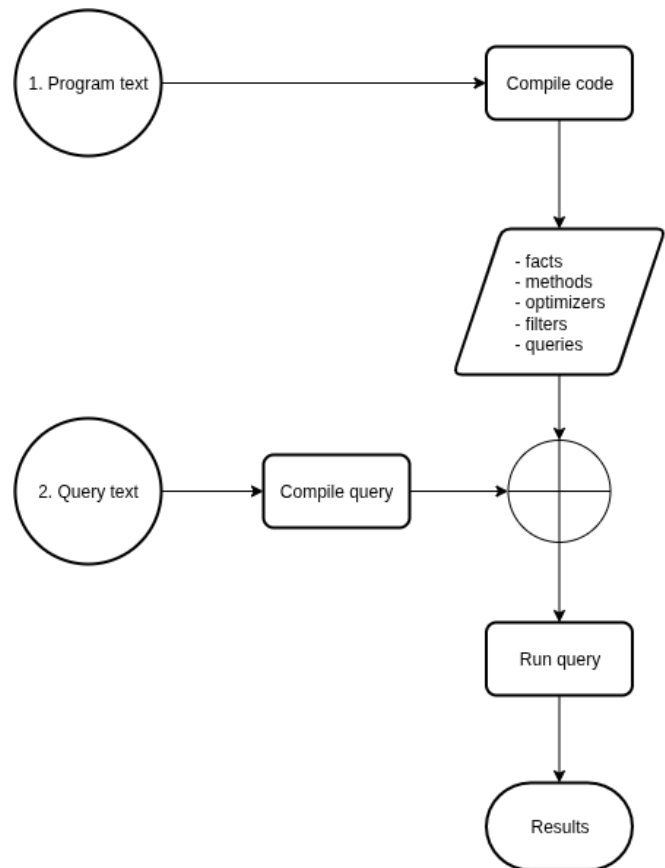


Figure 3: Performing a compilation of a program and subsequently submitting a query

RUNTIME PROCESSING

Any statement is evaluated sequentially, and the context is passed from one statement's invocation to another. The execution not only processes each expression one after the other, but individually every argument of the expression recursively. The context is enriched during the processing of each expression with a depth-first approach.

If an argument of a function call is a nested function call as well, it is going to be performed first, and the resulting context is passed to the hosting function call. But how is it possible to match the invoking argument choices with the target function head? Each call is defined with the context and with the invoking mask. LEM is the basic meta-structure in Kubrick, and it is adopted for defining facts and everything else in the language. The mask is a LEM that mirrors the structure of the call expression, but instead of containing primitives, it maps just variable identifiers which are stored in the corresponding reticolo's cluster. The process from caller to function and return, could be summarized as:

- Caller statement is performed on all its arguments.
- A mask of variables is defined, and the associated context has completed the execution of all arguments.
- The tuple <MASK, CONTEXT> is then passed to the blender.
- The blender searches for facts and methods. All results are appended in the same response.
- Matching methods are those that their HEAD is compatible with the MASK. Their PREMISE is executed by assigning the local-method variables to those passed with the MASK.
- The process repeats until no further call is requested.

In fact, each expression is a query that is performed when the related head matches an invocation, with the difference that the query context (the set of variables and respective values visible at that step in the process) is passed back to the invoker through the head interface. The same procedure is applied even a more granular level, when the system is inspecting a method call the arguments may be fixed constant values, materialized variables or brand-new ones, or even function calls that should be evaluated to get the single argument. So, the engine will process all arguments in the same way it will do with methods' bodies and user triggering queries.

OR/AND COMBINERS

All evaluations at any step are governed by two simple logical operators (OR, AND), which are also represented as LEM sub-types. In the language, those operators are indicated by semicolon “;” for OR, while the space “ ” and the comma “,” stands for AND. A space could be used to split the arguments of a predicate, while comma is a delimiter for predicates, combined by AND logic.

An invoker like (area-triangle 3 4) will be represented as a tuple which is a sequence of elements linked with AND operator. Similarly, the statement:

```
book author → Author name → BookName,
author id → Author name → AuthorName
```

Indicates an AND relation between two callers, the **book** and the **author** one. As an AND, a strict join is performed between the two respective resulting reticolo.

On the other hand, it is possible to define OR relations that are semantically equivalent to the sum of their results.

```
likes Subject Object; prefers Subject Object
```

This is an OR relation between two invokers (**likes**, **prefers**) and the corresponding reticolos are appended one after the other. Basically, a weak join is performed between the resulting reticolos. The OR relation has some constraints. The expressions involved in OR relations necessarily carry the same variables for being merged.

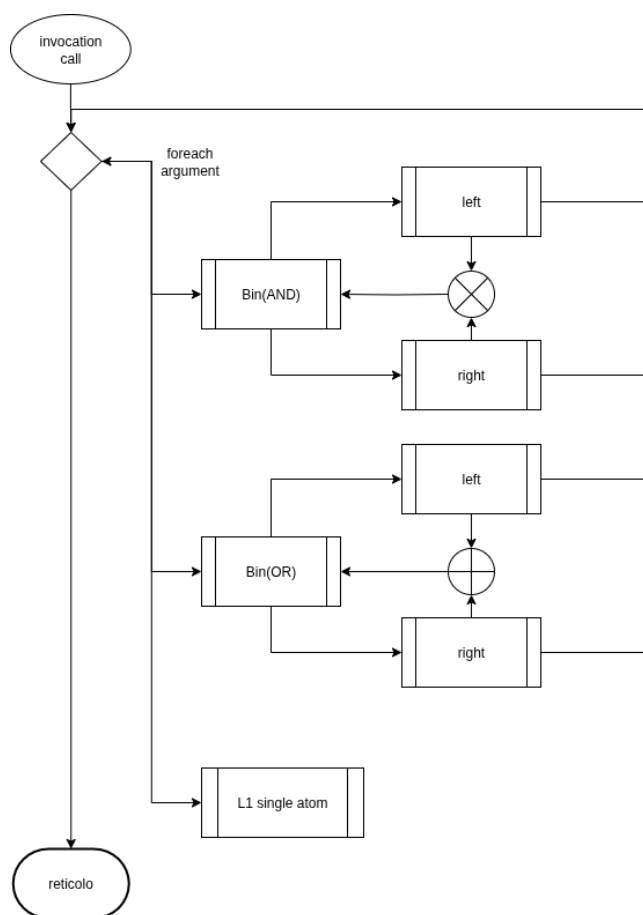


Figure 4: Workflow of an expression invocation call. Each element of the expression is computed. If the relation with other elements is an and then they will be (weakly) joined otherwise their values will be aggregated. The process repeats recursively for all elements.

The same process applied for the single argument and its entire predicate, is adopted for merging multiple methods that are compatible with the invoker request. Not only are methods performed, but also the database is used for grabbing information that matches facts. All clusters are merged according to the OR procedure, so they are summed together.

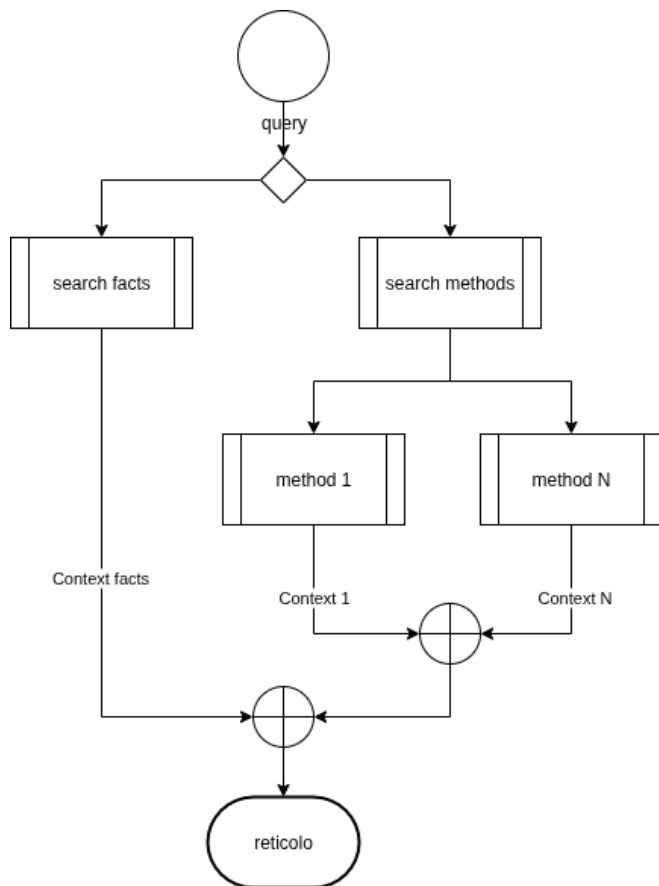


Figure 5: Single invocation workflow. The invoking call will search for facts and methods. All matching data and method output will be merged.

THE ENGINE

The entire software is written in Scala, a functional programming language that runs on the java virtual machine (JVM). The choice of Scala is driven by personal preferences accommodated by the sophisticated compiler, the type-system, and the ecosystem of functional libraries. Specific for this project, no dependency has been added other than common libraries for functional programming such as cats and fs2.

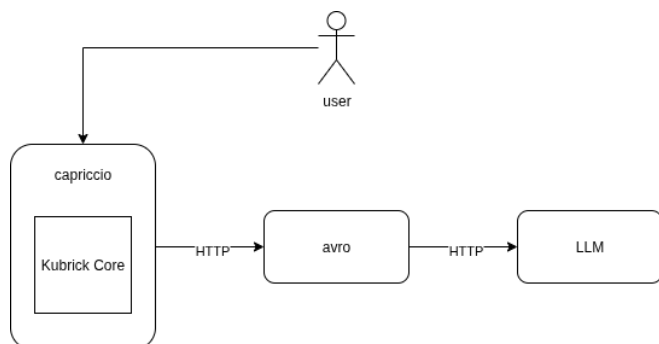


Figure 6: Module diagram of Kubrick deployment including 3 services (Capriccio, ARVO and the external LLM) and a static internal Kubrick's modules (Geronimo, Kube, Prelude and Reticolo).

TARGET DEPLOYMENTS

Kubrick engine is intended to target different environments to match possible customer demands. It is a multi-

target engine which is deployable in Hyperscalers infrastructure as “software as a service” (SaaS) by compiling with the scala-native backend, which is a wrapper of graal-vm a java virtual machine with justin-time capability at compile time. The JIT is an optimization process for accelerating the performance of programs. JIT is included in the Java runtime, and it is employed during runtime while the application is running. scala-native on the other hand, perform those optimizations at compile time, that means the startup of the application is much faster and does not need the usual warm-up. This is that deployment is indicated for cloud services where the application is shut down and restarted for optimal resource consumption.

The engine may work entirely in a web browser since it is also compiled in JavaScript, making it suitable to be integrated in single-page applications or progressive web apps (PWA). One target environment is also desktop applications, where the engine is compiled with a standard compiler.

CAPRICCIO: THE WEB USER INTERFACE

As mentioned in previous section about open authorship, when a programming system offers a fast feedback loop and the ability to consume AI features in an easy way, the interaction with the system may target a diverse type of users. Programmers and functional operators may have the ability to create complex workflows easily. There are already many examples, and some of them might be inspirational for Kubrick's user interface. Among them, I would mention spreadsheets, notebooks, and HTML inspectors. Spreadsheets are an example of shared substrates for all users. Anyone can modify data and formulas in a direct way. Notebooks are interesting because they allow us to split a complex program into a sequence of smaller steps by which the user may get immediate feedback on intermediate results. HTML inspectors allow to change the look & feel of a web page if the scripts govern it. The application includes all the Kubrick core modules directly compiled as JavaScript module executable by the browser. In this way, the computation is not delegated to centralized services but rather distributed among all users, a useful feature particularly adequate in case of combinatorial search. Mostly inspired by notebooks, I designed the UI as a sequence of cells:

- **Prompt cells.** The user writes instructions in English.
- **Code cells.** A Simple editor where generated code is visualized and modified. Code cells might be created by the user without inquiry about the LLM for that.
- **Result cells.** For each query present in the code, the query will be performed, and the results are shown there. For a multiple solution query, the user may fetch them one by one.

As a notebook, the user may add cells one after the other. The code placed in the cells above is visible to the cells below, so facts and methods are available for subsequent calls. The same applies also for prompts; all the accumulated code till the current prompt cell is submitted to the LLM as context. Of course, subsequent prompts can't change previous cells. When changes are applied, also the results are updated accordingly. Combined with the in-browser execution, it gives immediate feedback to the op-

erator while he's tweaking the program.

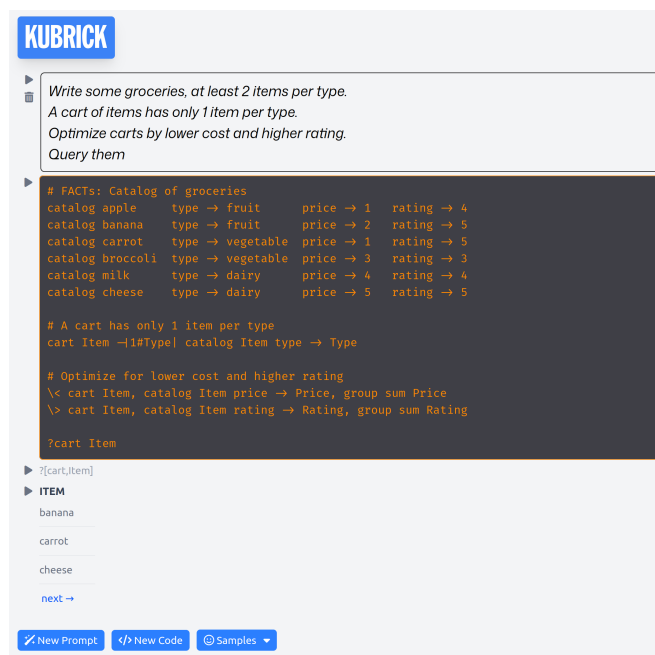


Figure 7: In a prompt cell, the user writes specifications. The AI tool transforms them into a Kubrick program which is then executed in the browser.

ARVO: THE LLM WRAPPER

ARVO is a separate runtime accessed by Capriccio for generating programs from user requests. Unlike all the rest of the software written for Kubrick, this system is entirely generated by AI tools. The seed prompt for it is the following: Build a Rust web server using Warp that exposes a POST endpoint at /.

1. The server loads a static prompt from prompt.txt at startup.
2. It accepts JSON requests with

```
{
  "prompt": "...",
  "history": [{
    "prompt": "...",
    "response": "..."
  }]
}
```

3. It composes a full prompt by appending the static prompt, previous interactions (if any), and the current prompt.
4. The server authenticates with an OAuth2 client credentials flow, caching the token and refreshing as needed.
5. It sends the composed prompt to a remote LLM endpoint (Gemini 2.5 Pro) using a specific payload structure.
6. The server parses the LLM response, extracting the first candidate's first part text, and returns it as plain text.

Unfortunately, Kubrick is not yet capable of applying those constraints for creating a general-purpose program and therefore the AI tool in use (GitHub Copilot) failed several times before generating a suitable application with the minimal required features. Without considering the quality of the code, the program was intended to be a prototype with low expectations. Though it is a low-risk adoption of AI for software development, the building of ARVO somehow reflected many of incurred problems already pointed

out regarding the effectiveness of AI tools for generating programs.

The system accepts Json payload in the HTTP request where the prompt submitted by the user is accompanied by all the attempted prompts and related generated code. In this way, the automaton can provide a contextualized answer that summarizes past interactions.

For generating programs, the LLM is requested to fulfill user demand by prompt engineering. Along with the user request, also a complete Kubrick language reference documentation is submitted to the AI tool (see Appendix B).

The LLM surprisingly can learn an entire brand-new language like Kubrick through single-shot learning.

INTEROPERABILITY WITH INFORMATION SOURCES

Software systems don't operate in vacuums. The ability to integrate with external information sources is a key for any programming system. The concise relational syntax allows full interoperability with different sources. Rather than operating on the internal database, why not externalize its relational semantics and transform premises into a data source specific query?

Besides recursion, there is no semantic gap with SparQL, for example. The Kubrick prototype has been adopted as an abstraction of the underlying RDF graph database. The idea here is to let the automatic generator focus just on the semantic aspects of omitting all data source-specific details for lowering the accidental complexity, which is one of my main goals.

REST APIS

The system is designed to consume external web services through REST APIs. Those services are available through bare HTTP protocol where the body content usually is a machine-readable format such as Json. For interpreting a function call as a web service invocation, it would be simply enough to specify a special tuple head like service:url with some fixed properties like url, method, and the payload. The service output could be automatically transformed into the engine's data structure since it is isomorphic with Json format. In this way, external services could be mapped into Kubrick language without shifting to a domain-specific interpreter.

Not only does Kubrick integrate external REST APIs, but it could be also accessible as a web service. Since the core structure is the named tuple, it would not be difficult to let clients encode queries directly in the request URI. For example, the function area-triangle could be invoked as:

```
GET /service/area-triangle?base=10&height=5
```

or

```
GET /service/area-triangle/10/5
```

returning a Json as a result.

MODEL CONTEXT PROTOCOL (MCP)

MCP is a service discovery protocol specifically introduced for AI tools. The characteristic of MCP is to pair the service specifications (URL, parameters, payload, etc.) with a description in natural language, so an LLM can choose the most appropriate service for a specific goal. Indeed, service discovery techniques are not new at all, but MCP got traction just because it is directly accessible for LLMs. Being the service list just a Json file, it is immediately incorporated into Kubrick data storage and then wrapping services as common REST services is just one step away.

COMBINATORICS IN KUBRICK

The term combinatorial problems refer to a broad range of problem classes that involve finding a solution from a finite set of possibilities. Each solution is distinct and countable, as they exist within a discrete solution space. These problems can be categorized as:

- **Decision problems.** Answering “*Is there a solution that satisfies the given constraints?*”
- **Optimization problems.** Seeking the best solution according to some criteria.
- **Counting problems.** Determining the number of solutions that meet the constraints.

These categories form a hierarchy: an optimization problem can only be formulated if a solution exists, and solutions can be enumerated, which leads to the counting family of problems.

An example of a decision problem is the graph coloring problem, which asks whether the vertices of a graph can be colored with a given number of colors so that no two adjacent vertices share the same color. The problem can be turned into an optimization problem by minimizing the number of colors used, or into a counting problem by asking how many solutions exist with a given number of colors.

To describe the phases of solving a combinatorial problem, I use a taxonomy like that of answer set programming (ASP):

GROUNDING

Grounding is the process of creating a superset of candidate solutions from the original problem space. This involves aggregating items to satisfy certain compositions. These aggregations can be defined as sets (combinations) or sequences (permutations). This pre-processing step can be extremely demanding in terms of computational resources. The term combinatorial explosion describes how a small increase in problem size can cause some problems—especially counting problems—become intractable.

In Kubrick, the grounding phase is managed by the generators. They generate a stream of facts according to a given criterion. The criteria are a set of declarations (Repetitions and Partitions) over variables. Let’s assume we need to model a restaurant, where a dish is a restaurant’s offering:

```
# Italian Dishes
dish bruschetta
  course → appetizer price → 8.50
  rating → 4          origin → italian
dish caprese
  course → appetizer price → 9.00
  rating → 5          origin → italian
dish lasagna
  course → main      price → 15.00
  rating → 5          origin → italian
dish carbonara
  course → main      price → 14.00
```

```
rating → 4          origin → italian
dish tiramisu
  course → dessert   price → 7.50
  rating → 5          origin → italian
dish panna_cotta
  course → dessert   price → 7.00
  rating → 4          origin → italian
# German Dishes
dish kartoffelsalat
  course → appetizer price → 7.00
  rating → 4          origin → german
dish bretzel
  course → appetizer price → 5.00
  rating → 3          origin → german
dish schnitzel
  course → main      price → 18.00
  rating → 5          origin → german
dish sauerbraten
  course → main      price → 20.00
  rating → 4          origin → german
dish apfelstrudel
  course → dessert   price → 8.00
  rating → 5          origin → german
dish schwarzwälder
  course → dessert   price → 9.00
  rating → 5          origin → german
```

The restaurant needs to create different combinations of menus all consistent with their origin and with only one dish per course:

```
menu dish -|1^Origin, 1#Course|
dish Dish course → Course origin → Origin
```

The rule above differs from its canonical implication (syntax: CLAIM $\rightarrow$ PREMISE) by injecting special information regarding the generation of possible combinations. In particular, the variables of `Origin` and `Course` are interested in this phase: the `Origin` has a Partition constraint, while the `Course` is constraint in its Repetition. This generator definition (`1^Origin, 1#Course`) could be translated as:

take only one origin and one repetition of unique courses, at time for each solution

Two of the Solutions will be:

Solution 1		Solution 2	
dish	course	dish	course
bruschetta	appetizer	bretzel	appetizer
lasagna	main	schnitzel	main
tiramisu	dessert	apfelstrudel	dessert

PARTITIONS

With the syntax `min^variable^max` it is intended that the values of variable are partitioned in way that each solution will hold at least min occurrences of variable unique values (independently by whether they are repeated or not) till a

maximum of max. If min is specified but max isn't, then max is intended to be set with the min value, and vice-versa. In the example above, the Origin has minimal and maximal occurrences set to one. Therefore, a solution may contain either italian or german dishes, not both.

Generators might be combined in an arbitrary way, like in the following sample, where we have a set of students with different skills and projects with different complexity. We want to form teams for each project with max two students with required skills:

```
# Students with their skill level
student alice skill → beginner
student tom skill → beginner
student bob skill → advanced
student cynthia skill → advanced
student alissa skill → advanced

# Projects with their required skill level
project quantum skill → advanced
project tictac skill → beginner

# For each project, form teams of 2 students that match
→ skill level.
team Student Project → 1^Project, 2^Student | student
→ Student skill → Level, project Project skill → Level

# Show all possible team compositions across all generated
→ verses.
?team Student Project
```

There will be one solution for the tictac project with alice and tom. Three solutions for the quantum project (all combinations: $\text{binomial}(3, 2) = 3$).

REPETITIONS

Repetitions state how many unique instances of values carried by a variable must be used for each Solution. Above, the 1#Course defines that there must be strictly one occurrence of course values. The number of repetitions is configurable min#variable#max where min is the minimum number of unique instances of a variable accepted in a Solution. The omitted max or min follows the pattern applied for Partitions.

Generators can be concatenated one after another. In addition to the multiverse setup for menu, in cascade more versions of data could be generated. If the restaurant has wine in the cellar and what to enrich the menu offering with a bottle of wine:

```
wine brunello type → red
wine lagrein type → red
wine prosecco type → white
wine vermentino type → white
recommended_wine Wine -|1#Wine| wine Wine
```

FILTERS

The prune out Solutions based on criteria. If we want to show in the economy section of our offerings only those menus cheaper than 30€:

```
\ menu Dish, dish Dish price → price,
< (group sum Price) 30
```

A Premise prefixed by \ is a positive filter that lets pass those Solutions that succeed with the Premise. A negative filter is declared with !\ prefix and let pass those Solutions that fail. The following example is equivalent with previous one:

```
!\ menu Dish, dish Dish price → price,
≥ (group sum Price) 30
```

In the restaurant domain, if we need to properly associate the recommended_wine according to the type of main dish, we should exclude all combinations where a red wine is associated with fish or a white wine is paired with a meat course:

```
!\ menu Dish,
dish Dish course → main content → meat,
recommended_wine Wine, wine Wine type → white
!\ menu Dish,
dish Dish course → main content → fish,
recommended_wine Wine, wine Wine type → red
```

MULTI-OBJECTIVE OPTIMIZATION

There are 3 types of optimizations:

1. **Fully strict.** All criteria are strictly prescriptive. That means only solutions that maximize every criterion are passed.
2. **Hybrid strict/Pareto.** some criteria are strict others are Pareto driven. The Pareto optimizer allows you to select solutions that are optimal for all Pareto criteria.
3. **Fully Pareto.** Solutions that turn out to be optimal are collected (frontier) and flushed out when all solutions have been fetched.

In (1) every optimizer is independent from the others and since the solutions are incrementally generated on demand, the optimizer does not have to hold the entire history of solutions to perform its function. The strict optimizer just holds the best score which compares the new incoming solutions. In (2) and (3) the system will find the Pareto optimal set of solutions. The Pareto optimizer will consider the optimal optimizers all together. Unlike strict optimizers, the Pareto optimizer keeps track of the optimal set. When a new incoming solution will be fetched there are three possible cases:

- **If** it dominates (i.e.: the new one is better in all optimizations) one of the optimal sets, the dominant solution will be dropped in favor of the new dominating one.
- **Else if** it is better in at least one optimization, it will be added to the frontier.
- **Otherwise**, it is discarded.

The Pareto optimization will then fetch all solutions before flushing the frontier, differently from the strict optimizer that just filters in/out solutions if they are generated. Those considerations imply that in (1) setup the streamed solutions will be monotonic in respect of the optimization criteria, i.e.: the solution score will only increase, and the last flushed solution will be the best one. Instead, in (2) and (3) the streamed-out solutions will be already the optimal ones, and they will be flushed out only when the last solution has been generated.

The syntax for optimizers: `\OPTIMIZATION PREMISE`, where `OPTIMIZATION` is:

- `>>` strict maximization
- `<<` strict minimization
- `>` Pareto maximization
- `<` Pareto minimization

`PREMISE` is the function to maximize/minimize. In the restaurant example, if we need to just get the cheapest menu:

```
\<< menu Dish,
    dish Dish price → Price,
    group sum Price
```

If we need to optimize (Pareto) for lower price and higher rating:

```
\< menu Dish,
    dish Dish price → Price,
    group sum Price
\> menu Dish,
    dish Dish rating → Rating,
    group sum Rating
```

Pareto optimization will occur only when there are at least two Pareto optimizers, otherwise the optimizer would be treated as a strict optimizer.

Lazy computation Within this section we have found that each solution is basically a computation performed over a single version of the database. Less noticeable but equally important, is that the entire system is based on the solution streaming paradigm. The system applies to the concept of call-by-need typical functional programming. Each solution is computed only when requested by the client. This does not prevent fetching the entire solution space, but it is useful when only one or few solutions are needed because the system does not have to compute all solutions in advance. Graphically, it is evident in the solution section. If a next link appears, it means that the end of the available solution has not been reached, and the next one - if any - will be elaborated.

EPILOGUE

As any experimental and complex artifact, Kubrick is not exempt from errors and limited implementations, though I adopted engineering practices for writing a reliable system. Automated testing has been in place since its inception, and testing coverage is in line with production standards. But its sole purpose of investigating and providing proof of what it is possible to do, did not give the traction to care properly about all incidents that might happen on playing with Kubrick.

It is difficult to predict the future of this system and now I can't affirm what will be done next, though the list of improvements is endless. I will list a series of limitations below. Being currently very experimental, it has been impossible to get feedback from users since it does not offer yet a compelling expressiveness for all programming problems a causal programmer can test on this system. Moreover, the system is not publicly available, and it is property of my employer, this is why the system can't be used to a large audience of users, so that feedback can naturally come from.

LIMITATIONS

NEGATION

In Kubrick, negation is not yet implemented. A pillar of LP that is essential for expression is missing. Though the modularity of the core framework won't prevent a smooth integration of negation capabilities.

NAIVE COMBINATORIAL OPTIMIZATION

As the purpose of Kubrick is to show how combinatorial problems can be easily encoded, the actual implementation does not reflect the state of the art of the domain. Conflict driven algorithms implemented by popular libraries like clingo can efficiently perform in-memory searches for solutions. Since Kubrick is intended to leverage external services, it would be the case to externalize those computationally demanding tasks. This kind of problem is not part of this project at this stage.

DATA INTEGRATION

While it won't affect the current modules, there is no implementation for transforming data protocols like Json, XML or whatever into LEM types.

RELATION ALGEBRA: DIVISION

Kubrick does not offer the forall command available in Prolog, which basically succeeds only when all choices succeed. Again, it is an important expression for LP languages, and Kubrick will surely benefit from having it in the future.

COMPARISON WITH ANSWER SET PROGRAMMING

Kubrick and ASP are declarative paradigms for solving complex combinatorial search and optimization problems. In the program, valid solution properties are specified, and a solver finds one or more solutions. Though, Kubrick offers a high-level, specialized syntax for combinatorial problems that differs significantly from ASP:

- **Partition:** this Kubrick generator expresses how many unique instances (independently by their repetition) will be used in a single Solution. In ASP, this is not a native concept. In the team building example, `1^Project` tells the engine to solve the problem for quantum and tictac independently. ASP does not have a built-in, high-level primitive analogous to the Partition.
- **Repetition:** the rule `min#var#max` is a direct readable instruction: choose min/max repetitions of single instances in var. This is analogous to an ASP cardinality constraint (choice rules), but it's part of the generator's definition, making the intent clearer and the code more localized.
- **Pareto optimization:** ASP does not have Pareto but lexicographic optimization. With it, every objective is assigned a specific priority, and then optimizations work as the strict optimization in Kubrick. In Kubrick, such prioritized optimization could be defined in a single optimization by writing a proper optimization function that customizes any given preference in a single score. Achieving true Pareto optimization in ASP on the other hand requires specialized extensions; it is not built-in in the core.

EVALUATING KUBRICK USABILITY

Though many arguments have been provided on the merit of the principles that have guided the choices applied to Kubrick, an attentive reader may ask about the effectiveness of automatons compared to other existing languages. For that, I applied the same prompt in Figure 1, but only with the preamble *"Write a Python program for the problem:..."*. The AI generated 112 rows of Python code (Appendix C) readily executable that present all shortcomings automatons have been mentioned in the introduction:

- Bloating imperative glue-code immersed in loop controls.
- More difficult to read.
- Naive notion of multi-objective optimization leading to incorrect answers

The unconvinced reader may still complain that with proper tool (library) the automaton may reasonably give proper answers without writing ad-hoc methods itself. I would stretch this argument to its extreme by saying that by applying all expedients like the mentioned before, the system will ultimately become very similar to Kubrick.

BIBLIOGRAPHY

- Abdelsalam, Y., Peitek, N., Maurer, A.-M., Toneva, M., & Apel, S. (2025). How do humans and LLMs process confusing code? [Preprint]. arXiv. <https://arxiv.org/abs/2508.18547>
- Aref, M., Guagliardo, P., Kastrinis, G., Libkin, L., Marsault, V., Martens, W., McGrath, M., Murlak, F., Nystrom, N., Peterfreund, L., Rogers, A., Sirangelo, C., Vrgoc, D., Zhao, D., & Zreika, A. (2025). Rel: A Programming Language for Relational Data [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2504.10323>
- Badreddine, S., d'Avila Garcez, A. S., Serafini, L., & Spranger, M. (2022). Logic Tensor Networks. *Artificial Intelligence*, 303, Article 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Balog, M., Gaunt, A. L., Brockschmidt, M., Nowozin, S., & Tarlow, D. (2016). DeepCoder: Learning to Write Programs. [Preprint]. ArXiv, abs/1611.01989.
- Basman, A., Lewis, C., & Clark, C. (2018). The open authorial principle: Supporting networks of authors in creating externalizable designs. Unpublished manuscript submitted to Onward '18. <https://github.com/amb26/papers/blob/master/onward-2016/onward-2016.pdf>
- Basman, A., Tchernavskij, P., Bates, S., & Beaudouin-Lafon, M. (2018). An anatomy of interaction: Co-occurrences and entanglements. In *Programming'18 Companion - Conference Companion of the 2nd International Conference on Art, Science, and Engineering of Programming* (pp. 188–196). ACM. <https://doi.org/10.1145/3191697.3214328>
- Bougzime, O., Jabbar, S., Cruz, C., & Demoly, F. (2025). Unlocking the potential of generative AI through neuro-symbolic architectures: Benefits and limitations [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2502.11269>
- Chollet, F., Knoop, M., Kamradt, G., & Landers, B. (2024). ARC Prize 2024: Technical Report. Arcprize.org.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- Dijkstra, E. W. (1972). *On the foolishness of “natural language programming”*. The CS Department at the University of Texas at Austin.
- Dincbas, M. (1987). Extended unification algorithms for the integration of functional programming into logic programming. *The Journal of Logic Programming*, 4(3), 199–227. [https://doi.org/10.1016/0743-1066\(87\)90002-1](https://doi.org/10.1016/0743-1066(87)90002-1)
- Ellis, K., Wong, C., Nye, M. I., Sablé-Meyer, M., Morales, L., Hewitt, L. B., Cary, L., Solar-Lezama, A., & Tenenbaum, J. B. (2021). Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *International Conference on Programming Language Design and Implementation (PLDI)*.
- Frisendal, T. (2022, July 22). Say hello to Graph Normal Form (GNF). *DATAVERSITY*.
- Ge, Y., et al. (2024). Openagi: When LLM meets domain experts. *Advances in Neural Information Processing Systems*.
- Ivanova, A. A., Srikant, S., Sueoka, Y., Kean, H. H., Dhamala, R., O'Reilly, U.-M., Bers, M. U., & Fedorenko, E. (2020). Comprehension of computer code relies primarily on domain-general executive brain regions. *eLife*. <https://doi.org/10.7554/eLife.58906>
- Jakubovic, J., Edwards, J., & Petricek, T. (2023). Technical Dimensions of Programming Systems. *The Art, Science, and Engineering of Programming*, 7(3), Article 13.
- Kahneman, D. (2011). *Thinking, fast and slow*. Farrar, Straus and Giroux.
- Kautz, H. A. (2022). The third AI summer: AAAI Robert S. Englemore memorial lecture. *AI Magazine*, 43(1), 105–125. <https://doi.org/10.1002/aaai.12036>
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*. OpenReview.net. <https://openreview.net/forum?id=SJU4ayYgI>
- Li, Y., Parsert, J., & Polgreen, E. (2024). Guiding enumerative program synthesis with large language models. *Proceedings of the ACM on Programming Languages*, 8(POPL), Article 13.
- Neward, T. (2006, June). The *Vietnam of Computer Science*.
- Ouellette, S. (2024). Towards efficient neurally-guided program induction for ARC-AGI. [Preprint]. arXiv preprint arXiv:2411.17708. <https://arxiv.org/abs/2411.17708>
- Saxena, S. (2025, September 6). Google launches Mangle: A new language that could change how we work with databases. *Peerlist*.
- Shi, K., Dai, H., Ellis, K., & Sutton, C. (2022). CROSS-BEAM: Learning to search in bottom-up program synthesis. [Preprint]. arXiv preprint arXiv:2203.10452.
- Trinh, T. H., Wu, Y., Le, Q. V., He, H., & Luong, T. (2024). Solving olympiad geometry without human demonstrations. *Nature*, 625(7995), 476–482. <https://doi.org/10.1038/s41586-023-06747-5>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*.
- Vijayakumar, A. K., Batra, D., Mohta, A., Jain, P., Polozov, O., & Gulwani, S. (2018). Neural-guided deductive search for real-time program synthesis from examples. In *Advances in Neural Information Processing Systems*. Retrieved from <https://arxiv.org/abs/1804.01186>
- Wang, W., Yang, Y., & Wu, F. (2022). Towards Data- and Knowledge-Driven Artificial Intelligence: A Survey on Neuro-Symbolic Computing [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2210.15889>
- Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., & Ji, H. (2024). Executable code actions elicit better LLM agents. arXiv. <https://arxiv.org/abs/2402.01030>

APPENDICES

APPENDIX B - LLM PROMPT ENGINEERING

Here, an extract of the prompt that is sent along with the user request for generating Kubrick programs. The total length of the prompt is 367 rows.

```
Your role is to act as an interpreter and solver for the programming language 'Kubrick'.
The following is a detailed, structured specification of this language.
Internalize these rules to accurately process and solve problems presented in Kubrick.
# Core Data Types
## ATOM types
You can define data in form of tuples (relations) of atom types:
- integer
- float
- boolean (true, false) without backtick or any wrapping char.
- string
* single word starting with a lowercase letter or an underscore. Can contain
uppercase letters, numbers, underscore symbols but not minus.
* quoted string: Text enclosed in double quotes ("). Example: "this is a sentence".
* there is no other way to define a string.
## VARIables
Placeholders for data.
- syntax: A single word starting with a capital letter, optionally followed by letters,
numbers, or underscores. Case-sensitive. Examples: `X`, `MyVar`, `Item_ID`.
- scope: A variable is local to the PREMISE in which it is defined.
* its name is not passed into a METHOD, instead the passed colum is renamed according
to the METHOD's CLAIM corresponding VARs.
- Free Variable (_): A special placeholder that matches any value but does not bind or
store it.
## TERM
The union of ATOM and VAR is called TERM.
# LEM[T] data types
LEM is a recursive, abstract data structure used for all data representation, including
facts, queries, and method definitions.
In this documentation, interpret ([T]) as a generic type like in scala language.
The data structure LEM is used for input/output in the storage system. It is the union
of:
- L1[T]
: single object wrapper. Implicitly used for any single `TERM`.
- PAIR[T] : key/value association. syntax: Lem[T] → LEM[T]
- SIC[T]
: ordered sequence of LEM[T]
- CHOICE[T]: unordered set of LEM[T]
The SIC is a dual data type: It implements sequence and dictionary (when PAIRS are
included) interfaces.
# Comments
A commented line starts with an hash "#". There is no multi-line comments.
Do not comment inside a METHOD. Do not comment inside a PREMISE.
# FACTs (database)
- set of LEM[ATOM] and represents the stored data of the program.
- no VAR allowed.
- in SIC[ATOM] atoms are separated by space.
- SIC[ATOM] might contain other SIC like: 'p attr → [1 2 3]' the 'attr' is valued with a
SIC of 3 integers.
- PAIRs represent property name/value
* syntax:(name → value).
- This is an example of SIC[ATOM]:
car idcar type → sedan manufacturer → mazda name → "3"
- CHOICEs are separated by semicolon:
car idcar color → red;blue;green
Above, the 'color' attribute's value is a CHOICE[ATOM] with 3 options (red, blue and
green) separated by semicolon. That means there 3 identical cars with different
color(Read QUERY section):
?car Id color → Color
```

```
# returns:
Id
Color
idcar red
idcar blue
idcar green
- if requested to generate FACTS, align the attributes starting all at the same textual
column so it is easier to read them. Double-check FACTs alignment.
...
```

APPENDIX C - PYTHON PROGRAM FOR GROCERY OPTIMIZER

```
import itertools
from collections import namedtuple

GroceryItem = namedtuple('GroceryItem', ['name', 'type', 'price', 'rating'])

GROCERIES = {
    'Fruit': [
        GroceryItem(name='Organic Bananas', type='Fruit', price=1.50, rating=4.5),
        GroceryItem(name='Standard Apples', type='Fruit', price=2.20, rating=4.0),
        GroceryItem(name='Strawberries', type='Fruit', price=3.00, rating=4.8),
    ],
    'Dairy': [
        GroceryItem(name='Organic Milk', type='Dairy', price=4.00, rating=4.9),
        GroceryItem(name='Store-Brand Yogurt', type='Dairy', price=0.80, rating=3.8),
    ],
    'Bakery': [
        GroceryItem(name='Sourdough Bread', type='Bakery', price=3.50, rating=4.7),
        GroceryItem(name='Whole Wheat Loaf', type='Bakery', price=2.50, rating=4.2),
    ],
    'Meat': [
        GroceryItem(name='Chicken Breast', type='Meat', price=7.00, rating=4.6),
        GroceryItem(name='Ground Beef', type='Meat', price=5.50, rating=4.3),
    ],
}

def analyze_all_carts():
    """Generates all possible carts, calculates their stats, and returns a list.
    A cart must contain exactly one item from each type."""
    item_lists_per_type = GROCERIES.values()
    all_possible_cart_combinations = itertools.product(*item_lists_per_type)
    analyzed_carts = []
    for cart_items in all_possible_cart_combinations:
        total_cost = sum(item.price for item in cart_items)
        total_rating = sum(item.rating for item in cart_items)
        score = total_rating / total_cost if total_cost > 0 else 0
        cart_data = {
            'items': cart_items,
            'total_cost': total_cost,
            'total_rating': total_rating,
            'score': score,
        }
        analyzed_carts.append(cart_data)
    return analyzed_carts

def main():
    """The main function to run the interactive query program."""
    print("Welcome to the Grocery Cart Optimizer!")
    print("Generating and analyzing all possible carts ... ")
    all_carts = analyze_all_carts()
    if not all_carts:
        print("Could not generate any carts. Check your GROCERIES data.")
        return
```

```

print(f"Analyzed {len(all_carts)} possible cart combinations.")
while True:
    print("\n=== Query Menu ===")
    print("1. Show the BEST VALUE cart (highest score)")
    print("2. Show the CHEAPEST cart")
    print("3. Show the HIGHEST-RATED cart")
    print("4. List top 5 best value carts")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")
    if choice == '1':
        best_cart = max(all_carts, key=lambda cart: cart['score'])
        print_cart_details(best_cart, title="Best Value Cart (Highest Score)")
    elif choice == '2':
        cheapest_cart = min(all_carts, key=lambda cart: cart['total_cost'])
        print_cart_details(cheapest_cart, title="Cheapest Cart")
    elif choice == '3':
        highest Rated cart = max(all_carts, key=lambda cart: cart['total_rating'])
        print_cart_details(highest Rated cart, title="Highest-Rated Cart")
    elif choice == '4':
        top_5_carts = sorted(all_carts, key=lambda cart: cart['score'], reverse=True)[:5]
        print("\nListing Top 5 Best Value Carts... ")
        for i, cart in enumerate(top_5_carts, 1):
            print_cart_details(cart, title=f"Rank #{i} Cart")
    elif choice == '5':
        print("Thank you for using the Grocery Cart Optimizer. Goodbye!")
        break
    else:
        print("Invalid choice. Please enter a number between 1 and 5.")

if __name__ == "__main__":
    main()

```